Answer layout (every calc)

Formula, then numbers substituted, then result (4 dp), then 1 line of meaning

Check lines = practice data with known answers, NOT exam answers. Same result = method right

State convention: ÷ n, log2, labels ±1. Setup: [SHIFT MENU] Input/Output 2

Fraction on screen? [SHIFT =] gives decimal. Store: [STO (−)] = A, use [ALPHA (−)]

Mean and variance (÷ n)

1) $\mu = \Sigma x \div n$

2) Table: x | x − μ | $(x-\mu)^2$

3) $\sigma^2 = \Sigma(x-\mu)^2 \div n$ (course: ÷ n, MLE)

Keys: [MENU] Statistics [=] [1], type each x [=], then [OPTN 3]: read $\bar{x}$ and $\sigma^2 x$

Trap: $s^2 x$ divides by n − 1, do NOT use. New data: [OPTN 2 2] clears

Check: −1,1,2,2,4: μ = 1.6, $\Sigma(x-\mu)^2$ = 13.2, σ^2 = 2.64

Write: μ = (sum)/n = ..., σ^2 = (sum of squares)/n = ... (MLE, ÷ n)

Gaussian naive Bayes: fit

1) Split the rows by class. Prior P(Ck) = nk ÷ n

2) Per class, per feature: μ and σ^2 (recipe above, ÷ nk)

3) μk = (μ1, μ2); Σk = [$\sigma1^2$ 0; 0 $\sigma2^2$] (diagonal)

Trap: use only that class's rows. Off-diagonals 0 because naive = features independent given class

Full (not naive) Σ: cov(x,y) = (1/n)Σ(x−μx)(y−μy) off-diagonal

Write: for each class μk vector, Σk matrix, prior

Gaussian naive Bayes: classify a point

1) Per class, per feature: $p = 1/\sqrt{(2\pi\sigma^2)} \times e^{(-(x-\mu)^2/(2\sigma^2))}$

Keys: e^ is [SHIFT ln]; put the whole exponent in brackets

2) score_k = P(Ck) × p(x1) × p(x2)

3) P(Ck|x) = score_k ÷ (score_0 + score_1); biggest wins

Table: class | p(x1) | p(x2) | prior | score

Check: μ0=(1.6,4.4), μ1=(3.4,2.6), all σ^2=2.64, priors 0.5, x=(2,4)

1/√(2π×2.64) = 0.2455. c0: p = 0.2382 each, score 0.0284

c1: p = 0.1694 each, score 0.0143. P(c0|x) = 0.6641: class 0

Trap: σ^2 (not σ) under the root; $2\sigma^2$ in the exponent

Bayes posterior (two branches)

1) Write P(A), P(B|A), P(B|¬A); P(¬A) = 1 − P(A)

2) top = P(B|A) × P(A)

3) P(B) = top + P(B|¬A) × (1 − P(A))

4) P(A|B) = top ÷ P(B)

Check: P(D)=.01, P(+|D)=.9, P(+|¬D)=.05: .009 ÷ .0585 = 0.1538

Trap: forgetting the ¬A branch in P(B)

MAP = argmax P(D|h)P(h); MLE = argmax P(D|h) (same if uniform prior)

Discrete naive Bayes + Laplace

1) Prior P(Ck) = nk ÷ n

2) Each feature: P(v|Ck) = count(v in class k) ÷ nk

3) Laplace: (count + 1) ÷ (nk + V), V = no. of possible values

Text: (count(w,C) + 1) ÷ (words in C + vocab size)

4) score_k = prior × product of step 2/3; biggest wins

Check tennis (sunny,cool,high,strong): yes 0.0053 < no 0.0206: No

Why Laplace: one zero count makes the whole product 0

Confusion matrix metrics

1) Find TP, FN (actual +) and FP, TN (actual −); read the row labels

2) P = TP/(TP+FP) R = TPR = TP/(TP+FN)

3) FPR = FP/(FP+TN) Spec = TN/(FP+TN) = 1 − FPR

4) Acc = (TP+TN)/all F1 = 2PR/(P+R) = 2TP/(2TP+FP+FN)

Check TP5 FN15 FP30 TN50: P .1429, R .25, FPR .375, Spec .625, Acc .55, F1 .1818

ROC points, invert fix, AUC

1) Table: threshold | TPR | FPR | point (FPR, TPR)

2) Plot x = FPR, y = TPR; add (0,0), (1,1); draw diagonal y = x

3) Points below diagonal (TPR < FPR): worse than random

4) Fix: invert decisions (use 1 − score): (FPR,TPR) to (1−FPR, 1−TPR)

5) AUC: sort by x, sum trapezoids (x2−x1)(y1+y2)/2

Check: TP5 FN15 FP30 TN50: (.375, .25) below; inverted (.625, .75)

Check (0,0),(.2,.6),(.5,.8),(1,1): .06 + .21 + .45 = 0.72

Write: inverted AUC = 1 − old; 0.5 random, 1 perfect

Bootstrap probability

1) P(not picked in 1 draw) = 1 − 1/n

2) P(never picked in n draws) = (1 − 1/n)^n, tends to 1/e = 0.368

3) P(in sample) = 1 − (1 − 1/n)^n, tends to 0.632

Check n=10: 0.6513; n=100: 0.6340

Keys: (1 − 1 ÷ 10) [x^■] 10 [=], then 1 − [Ans] [=]

Write: about 63.2% in bag, 36.8% out-of-bag (free validation)

CNN output size and parameters

1) O = (N + 2P − F)/S + 1; P=0,S=1: O = N − F + 1. Output O×O×K

2) 2×2 pool stride 2: O = N/2, depth same, 0 params

3) Params = K(F·F·C_in + 1), C_in = depth of the input

Table: layer | in | F, S, P, K | out | params

Check 32×32×3, 6 filters 5×5: 28×28×6, 6(75+1) = 456

Compare 2 configs: write both as formulas, (new − old), sign: + up, − down

Swap s1>s2: pool1 (p−s+1)/2 UP; pool2 (p−s1−2s2+3)/4, diff (s2−s1)/4 DOWN

conv1 f1(3s^2+1) DOWN; conv2 f2(f1s^2+1) UP; total f1($s1^2$−$s2^2$)(f2−3) UP

k-means: one iteration

1) Table: point | d to c1 | d to c2 | cluster

2) d = $\sqrt{((x1-x2)^2 + (y1-y2)^2)}$; 3 dp for near ties

3) Assign each point to the nearest centre

4) New centre = mean of its points ($\bar{x}$, $\bar{y}$)

5) Repeat. Converged when no assignment changes

Check (2,8) to (5,5): √(9+9) = √18 = 4.243

Check: C(6,4) to (4.6,8.2) 4.427 vs (9.4,6.8) 4.405: moves. Use 3 dp

Mean of a cluster: stats mode [OPTN 3] $\bar{x}$ (x then y, clear between)

Write: clusters, new centres, "changed / unchanged"

Entropy, information gain, Gini

H of ONE node [a, b]: n = a + b; p1 = a/n, p2 = b/n H = −p1 log2 p1 − p2 log2 p2; log2 x = [log] x [)] ÷ [ALPHA (−)] (A = log 2); 0 log 0 = 0 Table: class | count | p | log2 p | −p log2 p, then add the last column Check H(2,3): p .4, .6; log2 −1.3219, −.7370; .5288 + .4422 = 0.9710

1) Parent H(S) from the parent counts (recipe above)

2) Each branch v: H(Sv) from its own counts

3) H(S|A) = Σ (|Sv|/|S|) H(Sv) (weight by branch size)

4) IG = H(S) − H(S|A); split on the max IG

Gini = 1 − Σp^2; GiniGain = Gini(S) − Σ (|Sv|/|S|) Gini(Sv)

Table: branch | +, − | size/total | H

Gain ratio = IG ÷ SplitInfo, SplitInfo = −Σ (|Sv|/|S|) log2(|Sv|/|S|)

Check H(9,5) = 0.940, H(1,3) = 0.811, Gini(9,5) = 0.459

S[29+,35−] H .9937; A1 [21,5][8,30]: H|A1 .7278, IG .2659

Linear regression (1 feature)

1) $\bar{x}$, $\bar{y}$. Table: x | y | x−$\bar{x}$ | y−$\bar{y}$ | (x−$\bar{x}$)(y−$\bar{y}$) | $(x-\bar{x})^2$

2) $w1 = \Sigma(x-\bar{x})(y-\bar{y}) \div \Sigma(x-\bar{x})^2$; $w0 = \bar{y} - w1\,\bar{x}$

3) $\hat{y} = w0 + w1x$; $SSE = \Sigma(y-\hat{y})^2$; $SST = \Sigma(y-\bar{y})^2$

4) $R^2 = 1 - SSE/SST$; MSE = SSE/n

Check x=1,2,3,4 y=2,3,5,6: w1 = 7/5 = 1.4, w0 = 0.5

SSE 0.2, SST 10, R^2 0.98, MSE 0.05

GD step: $\partial w0 = (2/n)\Sigma(\hat{y}-y)$; $\partial w1 = (2/n)\Sigma x(\hat{y}-y)$; $w \leftarrow w - \alpha\partial$

Same data from w=(0,0), α=0.1: ∂ = (−8, −23.5), w = (0.8, 2.35)

Trap: compute both ∂ with the OLD w, then update both

Logistic regression (labels ±1)

1) z = w1x1 + w2x2 + b

2) p(+1) = σ(z) = 1/(1 + e^(−z)); p(−1) = 1 − p

3) odds = p/(1−p) = e^z; log-odds = z

4) Class: z > 0 (p > 0.5) gives +1, else −1

Check w=(0.8,−0.5), b=−1, x=(3,2): z = 0.4, p = 0.5987, odds 1.4918

Loss of one point = −ln p(true class): −ln 0.5987 = 0.5130

Boundary: w·x + b = 0 (p = 0.5); 1 feature: x* = −b/w

AdaBoost: one round (labels ±1)

0) Start: every weight w = 1/n. eg 10 pts: 0.1 each

1) ε = ADD the weights of the WRONG points. eg 3 × 0.1 = 0.3

2) α = ½ ln((1 − ε)/ε). eg ½ ln(0.7/0.3) = 0.4236 Keys: 0.5 × [ln] 0.7 ÷ 0.3 [)] [=]

3) Reweight: wrong w × e^α, right w × e^(−α) eg e^α = 1.5275, e^−α = 0.6547: wrong 0.1528, right 0.0655 Keys: e^α = [SHIFT ln] [Ans] [=]

4) "Renormalise": Z = sum of ALL new weights; each w ÷ Z eg Z = 3 × 0.1528 + 7 × 0.0655 = 0.9165 wrong 0.1528/0.9165 = 0.1667, right 0.0655/0.9165 = 0.0714

5) Final prediction: Σ α × (each learner's ±1), take the sign eg 0.4236(+1) + 0.2(−1) + 0.3(−1) = −0.0764: predict −1

Sanity: after step 4 wrong points hold exactly half (3 × 0.1667 = 0.5)

Trap: ε is a SUM of weights, not count/n (only equal in round 1)

Softmax

1) e^(z) for each class, 2) sum them, 3) p_k = e^(zk) ÷ sum

Check z=(1,2,3): sum 30.1929; p = .0900, .2447, .6652

Perceptron update (labels ±1)

1) Add x0 = 1. ŷ = +1 if w·x > 0 else −1

2) Right: no change. Wrong: $w \leftarrow w + \eta(y - \hat{y})x$

Check w=(0,0,0), x=(1,1,1), y=+1, η=.5: w·x=0, ŷ=−1, w = (1,1,1)

Neural net forward and one backprop gradient

1) Hidden z_j = σ(Σ w x); output ŷ = Σ w2_j z_j; J = $(y - \hat{y})^2$

2) δout = $2(\hat{y} - y)$; $\partial J/\partial w2_j$ = δout × z_j

3) δh_j = δout × w2_j × z_j(1 − z_j); $\partial J/\partial w1_ji$ = δh_j × x_i

4) $w \leftarrow w - \eta \times$ gradient

Check y=2, ŷ=1, z=.5, w2=1: δout = −2, δh = −2×1×.25 = −.5

Dense layer params = (inputs + 1) × outputs; 784-128-10: 100480 + 1290

Attention weights

1) score_i = q·k_i ÷ √dk 2) softmax the scores 3) out = Σ α_i v_i

Check q·k = 1,3,4, dk=4: .5,1.5,2; α .1220,.3315,.5465; v 10,2,6: 5.1618

Cosine similarity, Euclidean distance

cos = (a·b) ÷ (|a| |b|), |a| = $\sqrt{(\Sigma a^2)}$. (1,2),(2,1): 4/5 = 0.8

Euclid d = $\sqrt{\Sigma(ai - bi)^2}$. Distance 0 = same; cos 1 = same direction

fx-97SG X keys

log2(x) = [log] x [)] ÷ [log] 2 [)] (no log-base key); test log2 8 = 3

Faster: [log] 2 [)] [=] [STO] [(−)] puts log 2 in A, then log2 x = [log] x [)] ÷ [ALPHA] [(−)]

e^x = [SHIFT ln]. Decimal: [SHIFT =]. Last result: [Ans]

Store A: [STO (−)], recall [ALPHA (−)]

Stats: [MENU] Statistics [=] [1]; [OPTN 3] results; [OPTN 2 2] clear

Brackets: $e^{(-(x-\mu)^2/(2\sigma^2))}$, 1/(1+e^(−z)): close every bracket

How to answer

Explain: claim, reason (because), formula or example, so-what

Compare: what each does, pro and con each, when to use

Justify: choice, deciding property, what fails otherwise

Name key terms. ~1.7 min/mark; method marks count

Explainers

Why do we need a validation set? To choose models and hyperparameters (α, depth, epochs) and detect overfitting on unseen data. Training error is optimistic, so cannot pick models.

Split validation from the train or the test set? From the TRAINING set. Test is used once at the end; tuning on it leaks test info (data leakage), so the reported score is too optimistic.

Validation vs cross-validation? Hold-out: one split, cheap, high variance, val data never trained on. k-fold CV: train on k-1 folds, validate on 1, rotate, average; all data validated once, lower variance, k× cost.

When stratified or time-series CV? Stratified keeps class ratios per fold (imbalanced data). Time data: validate on a later period, no shuffling.

Why are d+1 params enough for binary logistic? Softmax K=2, divide by e^(w1·x+β1): p(1|x) = 1/(1+e^(-(w1-w2)·x-(β1-β2))). Only differences matter: w = w1-w2, β = β1-β2, so d+1; p(2|x) = 1-p(1|x).

Why is K(d+1) redundant? Adding the same (c, b) to all classes leaves softmax unchanged, so fix one class at 0: (K-1)(d+1) free params.

Why does a bootstrap sample hold ~63.2% of points? Missed in one draw w.p. 1-1/n, in n draws (1-1/n)^n → 1/e ≈ 0.368, so P(chosen) = 1-(1-1/n)^n ≈ 0.632.

What is the out-of-bag (OOB) set for? The ~36.8% not drawn are unseen by that model, so they give a free validation (OOB) error estimate.

ROC points below the diagonal: problem and fix? TPR < FPR, worse than random (AUC < 0.5): scores are inverted. Invert the classifier (use 1-score): (FPR, TPR) becomes (1-FPR, 1-TPR), AUC becomes 1-AUC.

Has k-means converged? Only if an assignment step moves no point, so centres stop moving. If any point switched or a centre moved: No, iterate again.

Why do total conv params rise when the big kernel moves to layer 2? Params = f(C_in k^2 + 1). Layer 1 has C_in = 3, layer 2 has C_in = f1 > 10, so big kernels cost more there: change = f1($s1^2$-$s2^2$)(f2-3) > 0.

Fitting, bias-variance, regularisation

Overfitting vs underfitting? Overfit: train error low, val high, fits noise (high variance). Underfit: both high, too simple (high bias); fix: more features or complexity, less regularisation.

How to reduce overfitting? More data or augmentation, simpler model, L1/L2 regularisation, pruning, dropout, early stopping, bagging.

Bias-variance trade-off? Error = bias^2 + variance + noise. More complexity lowers bias, raises variance; choose the complexity at minimum validation error.

How does regularisation help? Loss + α‖w‖ penalises big weights, so the fit is smoother: variance falls, bias rises a little. Larger α = simpler model.

L1 (Lasso) vs L2 (Ridge)? L2 αΣw^2 shrinks all weights smoothly, rarely to exactly 0. L1 αΣ|w| sets some w exactly 0 = feature selection, sparse model.

Learning rate too big or too small? Too small: tiny steps, slow. Too big: overshoots the minimum, loss oscillates or diverges.

Why gradient descent, not the closed form? Normal equation $(X^TX)^{-1}X^Ty$ costs ~O(d^3) and needs all data in memory; GD scales to big d, n and works where no closed form exists (logistic, NN).

Parameter vs hyperparameter? Parameters are learned in training (w, b). Hyperparameters are set before, chosen on validation (α, k, depth, η).

Metrics

Why is accuracy poor on imbalanced data? Predicting all-majority (99% negative) scores 99% yet finds no positives. Use precision, recall, F1, PR-AUC; class weights, resampling.

Precision vs recall, when each? Precision TP/(TP+FP) when false alarms are costly (spam filter). Recall TP/(TP+FN) when misses are costly (cancer screening, fraud).

Effect of raising the threshold? Fewer predicted positives: FP fall so precision usually rises, FN rise so recall falls.

Why is F1 a harmonic mean? F1 = 2PR/(P+R) is pulled to the smaller value, so P=1, R=0.01 gives F1 ≈ 0.02 (arithmetic mean 0.505). Both must be good.

What do ROC and AUC show? ROC = TPR vs FPR over all thresholds; diagonal random, top-left perfect. AUC = P(random positive scored above random negative); 0.5 random, 1 perfect.

ROC vs PR curve? FPR divides by the many negatives, so ROC looks optimistic when positives are rare. PR curve ignores TN, so it is more informative on imbalanced data.

Bayes and naive Bayes

Generative vs discriminative? Generative models p(x|C)P(C), can generate x (NB, GAN); ok with little data, strong assumptions. Discriminative models p(C|x) directly (logistic, trees, NN); better with much data.

Naive Bayes assumption and why? Features conditionally independent given the class: p(x|C) = Π p(xj|C). d easy 1-D estimates instead of one d-dim joint; Gaussian NB covariance is diagonal.

Why does NB work though independence is false? Only needs the right class to score highest (argmax), not exact probabilities. Fast, little data.

Why Laplace smoothing? An unseen word gives P(w|C) = 0 and the product zeroes the whole class. (count+1)/(total+|V|) keeps every probability > 0 and still sums to 1.

MLE vs MAP? MLE = argmax P(D|h), data only. MAP = argmax P(D|h)P(h), adds a prior. Uniform prior: MAP = MLE. MAP safer with little data.

Decision trees

What does entropy measure? Impurity of class labels, H = -Σ p log2 p: 0 for a pure node, max (1 bit for 2 classes) at 50/50. Gini 1-Σp^2 behaves alike.

What is IG and how does ID3 use it? IG = H(parent) - Σ (|Sv|/|S|) H(Sv), the entropy drop from a split. ID3 greedily splits on the max-IG attribute, recurses.

Why does IG favour many-valued attributes? An ID or date splits into tiny pure subsets, max IG but useless on new data. Gain ratio = IG / SplitInfo penalises many branches.

Why do trees overfit and how to stop it? They can split until every training row is right, fitting noise (high variance). Pre-prune (max depth, min leaf size), post-prune on validation, or use RF.

Ensembles

Why do ensembles work? Combining diverse models cancels independent errors. Base models must beat chance (each 45% accurate: more voters is worse).

Bagging vs boosting? Bagging: parallel models on bootstrap samples, equal vote, cuts VARIANCE (RF). Boosting: sequential, each focuses on past errors, α-weighted vote, cuts BIAS; sensitive to noisy labels.

Why does a random forest decorrelate trees? Each split tries a random feature subset (≈√d), so one strong feature cannot dominate every tree; averaging less correlated trees cuts more variance.

How does AdaBoost reweight? ε = sum of weights of wrong examples; α = ½ ln((1-ε)/ε); wrong × e^α, right × e^(-α), renormalise. Next learner focuses on hard cases; predict sign(Σ αt ht(x)).

Stacking vs blending? Stacking trains a meta-model on out-of-fold predictions of the base models; blending trains it on a held-out set (simpler, uses less data).

Neural networks

Why a non-linear activation? Else layers collapse to one linear map (W2W1x): no XOR, no curves.

Vanishing gradient and fixes? Backprop multiplies many derivatives (σ' ≤ 0.25), so early layers get ~0 gradient and stop learning. Fix: ReLU, He init, batch norm, skip connections, LSTM.

How does dropout reduce overfitting? Drops random units in training so none rely on others; like averaging many sub-nets. Test: all units, × keep prob α.

Batch vs SGD vs mini-batch? Batch: all data per update, stable, slow. SGD: 1 sample, fast, noisy zig-zag. Mini-batch: balance, the default.

CNN

Why CNN over FNN for images? FNN: a weight per pixel per neuron, ignores layout. CNN: local connectivity + weight sharing = far fewer params, translation invariance, feature hierarchy.

What is weight sharing? One filter slides over all positions: params = k×k×C_in per filter, not image size; finds a pattern anywhere.

Purpose of pooling? Downsample (2×2 halves H, W): less compute, robust to small shifts; no weights.

Effect of padding and stride? Out = (n + 2p - k)/s + 1. Padding keeps size and border info; bigger stride shrinks output and compute.

Why ResNet skip connections? Block outputs F(x) + x: learns only the residual; gradient flows straight through the + to early layers, so 100+ layers train.

RNN, attention, transformer

RNN vs LSTM? RNN gradient over T steps ~ W^T vanishes or explodes: forgets long context. LSTM forget/input/output gates + additive cell state keep gradients.

Why self-attention over RNN? Every token links directly to every other (path length 1), all positions in parallel, no vanishing through time. Cost O(n^2).

Why scale by √dk? q·k sums dk terms, spread grows with dk; √dk stops softmax saturating (tiny gradients).

Why multi-head? Each head has its own Q, K, V projections, so each learns a different relation (syntax, coreference); concat × W^O.

Why positional encoding? Attention is order-blind; added sin/cos position vectors give word order.

Why mask the decoder? Token t must not see the future it predicts: scores j > t set to -∞.

BERT vs GPT? BERT: encoder-only, bidirectional, masked words: understanding. GPT: decoder-only, causal, next token: generation.

Generative AI

GAN, VAE, diffusion? GAN: generator fakes from noise vs discriminator (real/fake), min-max. VAE: encoder gives μ, σ, z = μ + σε, decoder rebuilds. Diffusion: learn to remove noise step by step.

Why RLHF after fine-tuning? LLM = pre-train (self-sup.), SFT (sup.), RLHF. SFT copies demos, cannot rank answers; human rankings train a reward model, PPO maximises reward minus KL.

Unsupervised

k-means weaknesses? Must pick k; init-sensitive local optimum (restarts, k-means++); assumes round equal-size clusters; outliers; needs scaling.

How to choose k? Elbow: plot SSE vs k, pick the bend (SSE always falls). Silhouette s = (b-a)/max(a,b): pick k with highest average. Or domain knowledge.

k-means vs hierarchical? k-means: k in advance, fast, random init. Agglomerative: merge closest pair, dendrogram cut at any k, deterministic, O(n^2).

What does PCA do, why centre? Projects onto top-k orthogonal max-variance directions (covariance eigenvectors by λ). Centre so Σ = (1/n)X^TX; var explained = Σtop λ / Σλ.

Why embeddings, why cosine? One-hot is huge, sparse, all words equally far. Dense learned vectors put similar meanings close; cosine a·b/(‖a‖‖b‖) compares direction, ignores length.